1. K-Means Clustering

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

X = np.array([
    [1, 2], [2, 1], [3, 4],
    [5, 7], [6, 8], [7, 6],
    [8, 5], [4, 3]
])

kmeans = KMeans(n_clusters=3,
random_state=42)
labels = kmeans.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    marker='x'
)
plt.show()
```

2. Elbow Method

```
wcss = []

for k in range(1, 9):
    km = KMeans(n_clusters=k,
random_state=42)
    km.fit(X)
    wcss.append(km.inertia_)

plt.plot(range(1, 9), wcss, marker='o')
plt.show()
```

3. Hierarchical Clustering

```
from scipy.cluster.hierarchy import
dendrogram, linkage

Z = linkage(X, method='ward')

dendrogram(Z)
plt.show()
```

4. DBSCAN

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=1.5, min_samples=2)
db_labels = dbscan.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1],
c=db_labels)
plt.show()
```

5. Association Rule Learning

```
transactions = [
    {"Bread", "Milk", "Egg", "Butter",
"Salt", "Apple"},
    {"Bread", "Milk", "Egg", "Apple"},
    {"Bread", "Milk", "Butter", "Apple"},
    {"Milk", "Egg", "Butter", "Apple"},
    {"Bread", "Egg", "Salt"},
    {"Bread", "Milk", "Egg", "Apple"}
]

def support(itemset, transactions):
    return sum(1 for t in transactions if
itemset.issubset(t)) / len(transactions)

def confidence(X, Y, transactions):
    return support(X.union(Y),
transactions) / support(X, transactions)

print(support({"Bread"}, transactions))
print(confidence({"Bread"}, {"Milk"},
transactions))
```

6. Artificial Neuron

```
import numpy as np

def artificial_neuron(inputs, weights,
bias=0):
    y_sum = np.dot(inputs, weights) +
bias
    return y_sum

x = np.array([1, 0, 1])
w = np.array([0.5, -0.6, 0.2])

print(artificial_neuron(x, w, bias=0.1))
```

7. Activation Functions

```
import math

def identity(x):
    return x

def step(x):
    return 1 if x >= 0 else 0

def threshold(x, theta=0.5):
    return 1 if x >= theta else 0

def relu(x):
    return max(0, x)

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

def bipolar_sigmoid(x):
    return (2 / (1 + math.exp(-x))) - 1

def tanh_n(x):
    return math.tanh(x)
```

8. McCulloch Pitts Neuron (OR Gate)

```
def mcp_neuron(x1, x2, w1=1, w2=1,
threshold=1):
    y_sum = x1*w1 + x2*w2
    return 1 if y_sum >= threshold else 0
for x1 in [0,1]:
    for x2 in [0,1]:
        print(x1, x2, "->",
mcp_neuron(x1, x2))
```

9. Perceptron

```
import numpy as np
class Perceptron:
    def __init__(self, lr=0.1,
epochs=10):
        self.lr = lr
        self.epochs = epochs

    def activation(self, x):
        return 1 if x >= 0 else -1
    def fit(self, X, y):
        self.weights =
np.zeros(X.shape[1])
        self.bias = 0
        for _ in range(self.epochs):
            for i in range(len(X)):
                linear = np.dot(X[i],
self.weights) + self.bias
                y_pred =
self.activation(linear)
                update = self.lr * (y[i]
- y_pred)
                self.weights += update *
X[i]
                self.bias += update

    def predict(self, X):
        linear = np.dot(X, self.weights)
+ self.bias
        return self.activation(linear)
```

10. ADALINE

```
class Adaline:
    def __init__(self, lr=0.01,
epochs=20):
        self.lr = lr
        self.epochs = epochs
    def fit(self, X, y):
        self.weights =
np.zeros(X.shape[1])
        self.bias = 0
        for _ in range(self.epochs):
            for i in range(len(X)):
                y_sum = np.dot(X[i],
self.weights) + self.bias
                error = y[i] - y_sum
                self.weights += self.lr *
error * X[i]
                self.bias += self.lr *
error
    def predict(self, X):
        y_sum = np.dot(X, self.weights) +
self.bias
        return 1 if y_sum >= 0 else -1
```

11. Single Layer Feed Forward Network

```
from sklearn.linear_model import
Perceptron
from sklearn.datasets import
make_classification
from sklearn.model_selection import
train_test_split
from sklearn.metrics import
accuracy_score

X, y = make_classification(
    n_samples=200,
    n_features=2,
    n_redundant=0,
    n_clusters_per_class=1
)

X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.3
)

model = Perceptron(max_iter=1000,
eta0=0.1)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(accuracy_score(y_test, y_pred))
```

12. Multi Layer Perceptron (Backpropagation)

```
from sklearn.neural_network import
MLPClassifier

model = MLPClassifier(
    hidden_layer_sizes=(10,5),
    activation='relu',
    learning_rate_init=0.01,
    max_iter=500
)

model.fit(X_train, y_train)
```

13. Manual Backpropagation (XOR)

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

np.random.seed(1)
weights = np.random.rand(2,1)

def sigmoid(x):
    return 1/(1+np.exp(-x))

def sigmoid_derivative(x):
    return x*(1-x)

for epoch in range(5000):

    output = sigmoid(np.dot(X, weights))

    error = y - output

    adjustments = 0.1 * np.dot(
        X.T,
        error *
sigmoid_derivative(output)
    )

    weights += adjustments

print(output)
```

14. Deep Neural Network

```
model = MLPClassifier(
    hidden_layer_sizes=(64,32,16),
    activation='relu',
    learning_rate_init=0.01,
    max_iter=500
)
```

15. ICA

```
from sklearn.decomposition import FastICA

ica = FastICA(
    n_components=2,
    random_state=0
)

U = ica.fit_transform(X)
```

16. Autoencoder

```
from sklearn.neural_network import
MLPRegressor

autoencoder = MLPRegressor(
    hidden_layer_sizes=(2,),
    activation="relu",
    max_iter=5000
)

autoencoder.fit(X, X)
```

17. Active Learning

```
from sklearn.linear_model import
LogisticRegression

model =
LogisticRegression(solver="liblinear")

for i in range(5):
    model.fit(X[labeled], y[labeled])

    probs =
model.predict_proba(X[unlabeled])[:,1]

    query = np.argmin(np.abs(probs -
0.5))

    labeled = np.append(labeled,
unlabeled[query])
```

18. RBF Network

```
def rbf(x, c):
    return np.exp(
        -np.linalg.norm(x-c)**2 /
        (2*sigma**2)
    )
H = np.array([
    [rbf(x,c) for c in centers]
    for x in X
])
w = np.linalg.pinv(H) @ y
```

19. ECLAT

```
transactions = [
    {"A","B"},
    {"A","C"},
    {"B","C"},
    {"A","B"}
]
tid = {}

for i, t in enumerate(transactions):
    for item in t:
        if item not in tid:
            tid[item] = set()

        tid[item].add(i)
AB = tid["A"] & tid["B"]

print(len(AB))
```

20. Bagging

```
from sklearn.ensemble import
BaggingClassifier
from sklearn.tree import
DecisionTreeClassifier

bag = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=20,
    bootstrap=True
)

bag.fit(Xtr, ytr)
```

21. AdaBoost

```
from sklearn.ensemble import
AdaBoostClassifier

ada = AdaBoostClassifier(

estimator=DecisionTreeClassifier(max_dept
h=1),
    n_estimators=50,
    learning_rate=1.0
)

ada.fit(Xtr, ytr)
```

22. Gradient Boosting

```
from sklearn.ensemble import
GradientBoostingClassifier

gbm = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1
)

gbm.fit(Xtr, ytr)
```

23. Elastic Net

```
from sklearn.linear_model import
ElasticNet

model = ElasticNet(
    alpha=1.0,
    l1_ratio=0.5
)

model.fit(Xtr, ytr)
```

24. Q-Learning Agent

```
class SimpleAgent:

    def __init__(self, n_states,
n_actions):
        self.q_table = np.zeros(
            (n_states, n_actions)
        )

        self.alpha = 0.1
        self.gamma = 0.9
        self.epsilon = 0.1

    def choose_action(self, state):
        if np.random.rand() <
self.epsilon:
            return np.random.choice(
                len(self.q_table[state])
            )

        return
np.argmax(self.q_table[state])
```

25. Policy Evaluation

```
def policy_evaluation(policy, P, R,
                      gamma=0.9,
                      theta=1e-5):

    V = np.zeros(len(policy))

    while True:

        delta = 0

        for s in range(len(policy)):

            v = V[s]
            a = policy[s]

            V[s] = R[s][a] + gamma * sum(
                prob * V[next_state]
                for next_state, prob in
P[s][a]
            )

            delta = max(delta, abs(v -
V[s]))

        if delta < theta:
            break

    return V
```

26. Value Iteration

```
def value_iteration(P, R,
                    gamma=0.9,
                    theta=1e-5):

    V = np.zeros(len(P))
    while True:
        delta = 0
        for s in range(len(P)):

            v = V[s]

            V[s] = max(
                R[s][a] + gamma * sum(
                    prob * V[next_state]
                    for next_state, prob
in P[s][a]
                )
                for a in P[s]
            )

            delta = max(
                delta,
                abs(v - V[s])
            )
if delta < theta:
            break

    return V
```

27. TD Learning

```
def td_learning(env,
                episodes=500,
                alpha=0.1,
                gamma=0.9):

    V = np.zeros(
        env.observation_space.n
    )

    for ep in range(episodes):

        state = env.reset()[0]
        done = False

        while not done:

            action =
env.action_space.sample()

            next_state, reward, done, _,
_ = env.step(action)

            V[state] += alpha * (
                reward +
                gamma * V[next_state]
                - V[state]
            )

            state = next_state

    return V
```

28. Q Learning

```
def q_learning(env,
               episodes=2000,
               alpha=0.1,
               gamma=0.9,
               epsilon=0.1):

    Q = np.zeros(
        (env.observation_space.n,
         env.action_space.n)
    )

    for ep in range(episodes):

        state = env.reset()[0]
        done = False

        while not done:

            if np.random.rand() <
epsilon:
                action =
env.action_space.sample()
            else:
                action =
np.argmax(Q[state])

            next_state, reward, done, _,
_ = env.step(action)

            Q[state, action] += alpha * (
                reward +
                gamma *
np.max(Q[next_state]) -
                Q[state, action]
            )

            state = next_state

    return Q
```

1. Write a program to save a trained machine learning model using `joblib.dump()`.

```
# SAVE TRAINED MODEL USING JOBLIB

# Required libraries
from sklearn.datasets import load_iris
from sklearn.tree import
DecisionTreeClassifier
import joblib

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Create model
model = DecisionTreeClassifier()

# Train model
model.fit(X, y)

# Save model
joblib.dump(model, "iris_model.pkl")

print("Model saved successfully")
```

2. Write a Python program to load a saved model using `joblib.load()` and make predictions.

```
# LOAD SAVED MODEL AND MAKE PREDICTION

# Required libraries
import joblib
import numpy as np

# Load model
model = joblib.load("iris_model.pkl")

# Sample input
sample = np.array([[5.1, 3.5, 1.4, 0.2]])

# Prediction
prediction = model.predict(sample)

print("Predicted Class:", prediction)
```

3. Write a Tkinter program to create a GUI window with the title "Iris Flower Classification".

```
import tkinter as tk

# Create window
root = tk.Tk()

# Window title
root.title("Iris Flower Classification")

# Window size
root.geometry("400x300")

root.mainloop()
```

4. Write a Tkinter program to create labels and entry boxes for Sepal Length and Sepal Width.

```
# LABELS AND ENTRY BOXES

import tkinter as tk

root = tk.Tk()

root.title("Iris Flower Classification")

# Sepal Length
label1 = tk.Label(root, text="Sepal
Length")
label1.pack()

entry1 = tk.Entry(root)
entry1.pack()

# Sepal Width
label2 = tk.Label(root, text="Sepal
Width")
label2.pack()

entry2 = tk.Entry(root)
entry2.pack()

root.mainloop()
```

5. Write a Python program to create a button in Tkinter that displays a popup message when clicked.

```
# BUTTON WITH POPUP MESSAGE

import tkinter as tk
from tkinter import messagebox

# Function
def show_message():
    messagebox.showinfo("Message",
"Button Clicked")

# Create window
root = tk.Tk()

root.title("Popup Example")

# Button
button = tk.Button(
    root,
    text="Click Me",
    command=show_message
)

button.pack()

root.mainloop()
```

6. Write a Python code snippet to handle invalid numeric input using try-except ValueError.

```
# ERROR HANDLING USING TRY-EXCEPT

try:
    value = float(input("Enter a number:
"))
    print("Value:", value)

except ValueError:
    print("Invalid Numeric Input")
```

---

7. Write a program to check whether the file `iris_model.pkl` exists using the os module.

```
# CHECK FILE EXISTENCE

import os

# File check
if os.path.exists("iris_model.pkl"):
    print("Model file exists")

else:
    print("Model file not found")
```

---

8. Write a Python program to create four input fields using Tkinter Entry widgets.

```
# FOUR INPUT FIELDS USING ENTRY WIDGETS

import tkinter as tk

root = tk.Tk()

root.title("Iris Flower Classification")

# Input 1
entry1 = tk.Entry(root)
entry1.pack()

# Input 2
entry2 = tk.Entry(root)
entry2.pack()

# Input 3
entry3 = tk.Entry(root)
entry3.pack()

# Input 4
entry4 = tk.Entry(root)
entry4.pack()

root.mainloop()
```

---

9. Write code to predict the flower category using `model.predict()` for user-entered values.

```
# FLOWER PREDICTION USING MODEL

import joblib
import numpy as np

# Load model
model = joblib.load("iris_model.pkl")

# User values
sl = 5.1
sw = 3.5
pl = 1.4
pw = 0.2

# Prediction
prediction = model.predict([[sl, sw, pl,
pw]])

print("Predicted Class:", prediction)
```

---

10. Write a Python program to display the predicted flower name using `messagebox.showinfo()`.

```
# DISPLAY PREDICTED FLOWER NAME

import tkinter as tk
from tkinter import messagebox

# Create window
root = tk.Tk()

root.withdraw()

# Predicted flower
flower = "Setosa"

# Display result
messagebox.showinfo(
    "Prediction",
    "Predicted Flower: " + flower
)
```

---

11. Write a program to create a fixed-size Tkinter window using `geometry()` and `resizable()`.

```
# FIXED SIZE WINDOW
import tkinter as tk

# Create window
root = tk.Tk()

root.title("Fixed Window")

# Window size
root.geometry("400x300")

# Disable resizing
root.resizable(False, False)

root.mainloop()
```

---

12. Write a Python program that loads the Iris dataset and prints the target names.

```
# LOAD IRIS DATASET

from sklearn.datasets import load_iris

# Load dataset
iris = load_iris()

# Print target names
print("Target Names:")

print(iris.target_names)
```